Course Name:	**Database Systems**
Name :	**AMINA ASIF**
Project :	**University Course Registration Database Design**

Part 1: ERD / EERD Design

1.0 Entity Relationship Diagram (ERD)

The diagram below shows the complete ERD for the University Course Registration and Management System. Primary keys are underlined and bold; foreign keys are italicised.

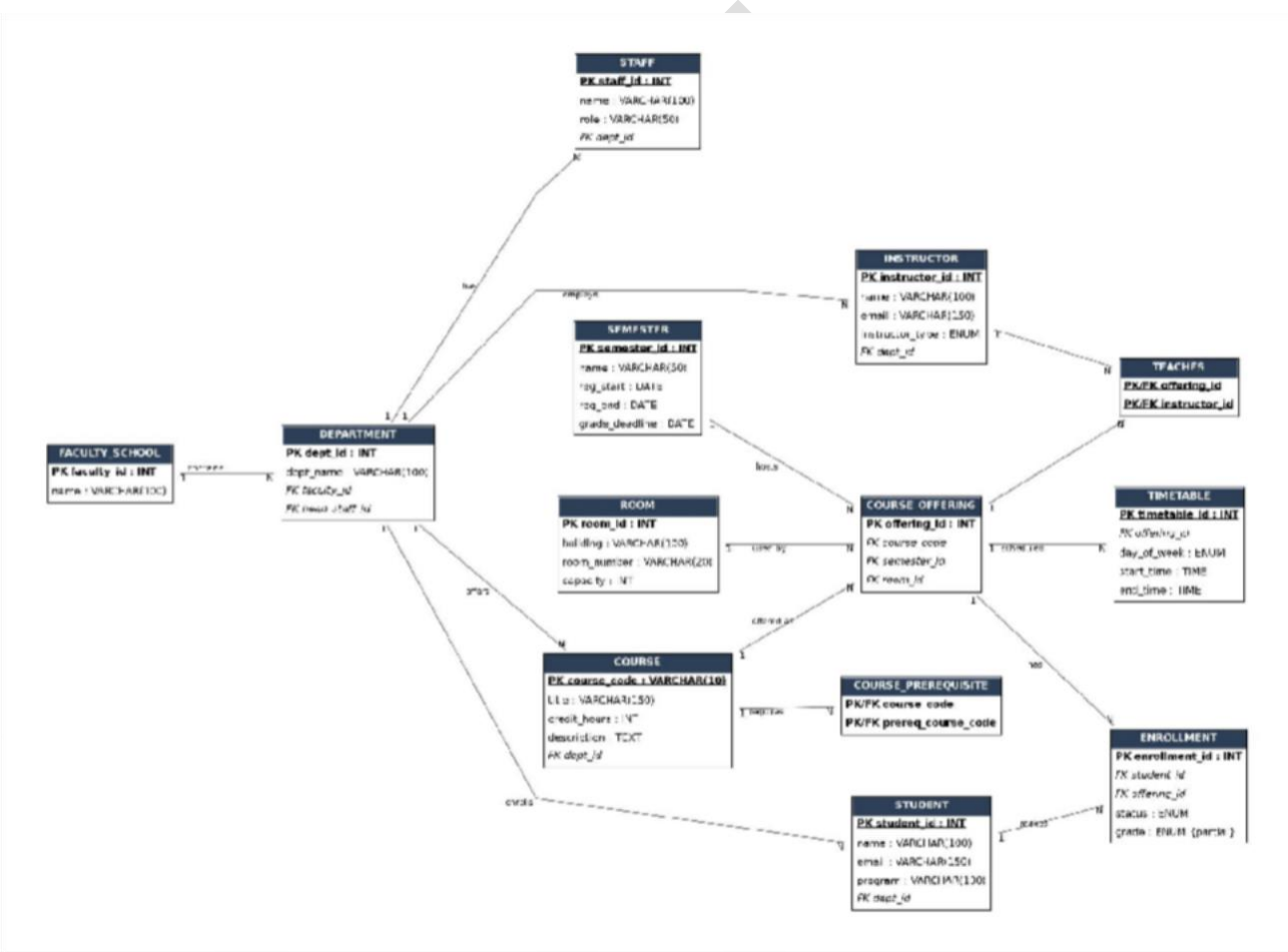


Figure 1: Complete ERD — University Course Registration and Management System

1.1 Entities and Their Attributes

The following entities were identified from the case study requirements. Primary keys are underlined; composite, multivalued, and derived attributes are noted.

Entity	Attributes	Notes
FACULTY_SCHOOL	faculty_id (PK), name	Top-level organizational unit
DEPARTMENT	dept_id (PK), dept_name, faculty_id (FK), head_staff_id (FK)	Belongs to a Faculty
STAFF	staff_id (PK), name, role, dept_id (FK)	Administrative staff of a dept
INSTRUCTOR	instructor_id (PK), name, email, type {Full-time | Visiting}, dept_id (FK)	Generalization: Full-time / Visiting (EERD)
STUDENT	student_id (PK), name, email, program, dept_id (FK)	Can register for multiple courses
COURSE	course_code (PK), title, credit_hours, description, dept_id (FK)	Offered by a department
COURSE_PREREQUISITE	course_id (FK), prereq_course_id (FK)	Recursive/associative relationship on COURSE
SEMESTER	semester_id (PK), name, reg_start, reg_end, grade_deadline	Temporal container for offerings
COURSE_OFFERING	offering_id (PK), course_id (FK), semester_id (FK), room_id (FK)	A course offered in a specific semester
TEACHES	offering_id (FK), instructor_id (FK)	M:N associative table — co-teaching allowed
ENROLLMENT	enrollment_id (PK), student_id (FK), offering_id (FK), status, grade	grade is partial — derived when submitted
ROOM	room_id (PK), building, room_number, capacity	Capacity constraint enforced at app layer
TIMETABLE	timetable_id (PK), offering_id (FK), day_of_week, start_time, end_time	Prevents scheduling conflicts

1.2 Relationships and Cardinalities

Relationship	Entities	Cardinality	Participation
contains	FACULTY_SCHOOL → DEPARTMENT	1 : N	Total on DEPARTMENT (every dept belongs to a faculty)
employs	DEPARTMENT → INSTRUCTOR	1 : N	Partial on INSTRUCTOR (visiting may be dept-less)
enrolls	DEPARTMENT → STUDENT	1 : N	Total on STUDENT
offers	DEPARTMENT → COURSE	1 : N	Total on COURSE
has (staff)	DEPARTMENT → STAFF	1 : N	Total on STAFF
offered as	COURSE → COURSE_OFFERING	1 : N	Partial on COURSE (not every course offered each sem)
hosts	SEMESTER → COURSE_OFFERING	1 : N	Total on COURSE_OFFERING
used by	ROOM → COURSE_OFFERING	1 : N	Partial on ROOM
scheduled in	COURSE_OFFERING → TIMETABLE	1 : N	Total on TIMETABLE
assigned to (TEACHES)	INSTRUCTOR ↔ COURSE_OFFERING	M : N	Partial both sides (co-teaching allowed)
makes	STUDENT → ENROLLMENT	1 : N	Partial on STUDENT
has (enrollment)	COURSE_OFFERING → ENROLLMENT	1 : N	Partial on COURSE_OFFERING
requires (prerequisite)	COURSE → COURSE (self)	M : N	Partial on both

1.3 EERD: Specialization of INSTRUCTOR

The INSTRUCTOR entity is generalized into two sub-types using disjoint, partial specialization:

- FULL_TIME_INSTRUCTOR — adds: office_number, employee_id, salary_grade
- VISITING_INSTRUCTOR — adds: home_institution, contract_end_date

Constraint: Disjoint (d) an instructor is either full-time or visiting, not both. Participation: Partial not every instructor must be categorized in the sub-type tables (e.g. adjunct cases).

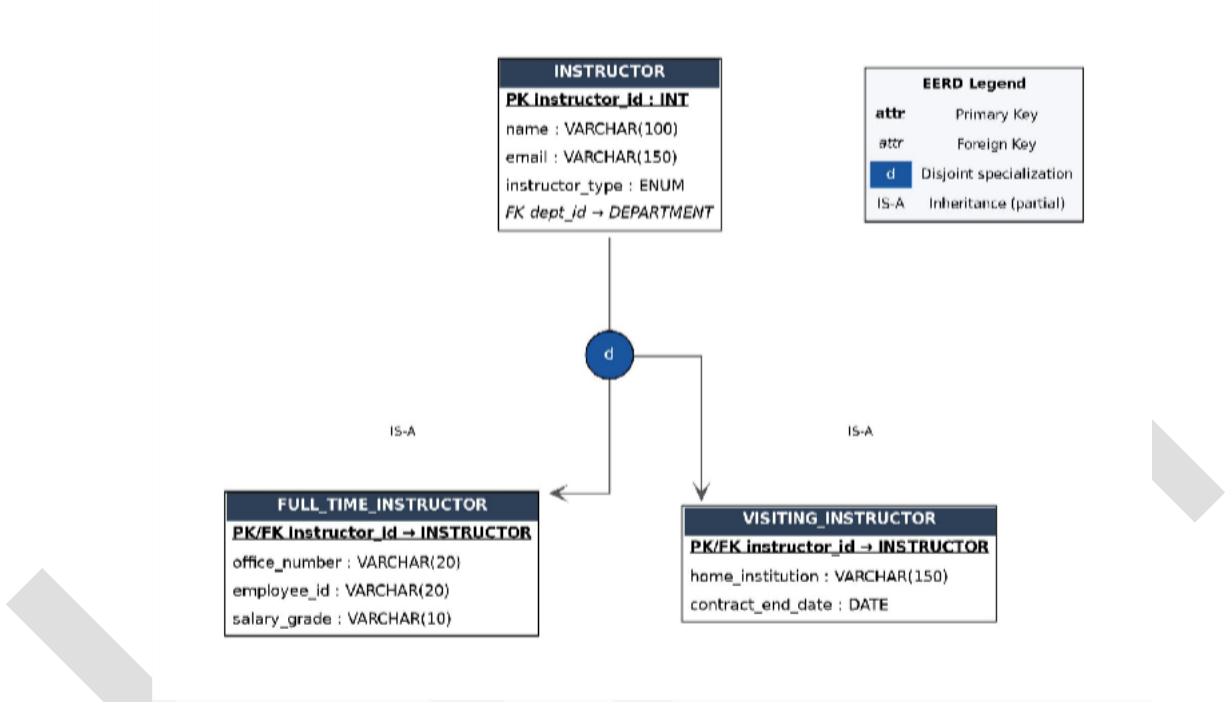


Figure 2: EERD — Disjoint Specialization of INSTRUCTOR into Full-time and Visiting subtypes

Part 2: Normalization (UNF → 1NF → 2NF → 3NF)

2.1 Unnormalized Relation (UNF)

Starting from a flat, "real-world" enrollment report that a university might produce, containing all data in one large table:

ENROLLMENT_REPORT (UNF):

enrollment_id, student_id, student_name, student_email, student_program, dept_id, dept_name, faculty_id, faculty_name, offering_id, course_code, course_title, credit_hours, course_description, semester_id, semester_name, reg_start, reg_end, room_id, building, room_number, capacity, instructor_id, instructor_name, instructor_email, instructor_type, day_of_week, start_time, end_time, grade, status, {prerequisites: [course_code, course_title]}

Problems with this UNF relation:

- Repeating group: prerequisites is a multivalued attribute (violates 1NF atomicity).
- Massive data redundancy: student info repeats for every course they take.
- Update anomaly: changing dept_name requires updating every row for that department.
- Insertion anomaly: cannot add a course with no students enrolled.
- Deletion anomaly: deleting the last enrollment removes the course's data.

ENROLLMENT_REPORT (UNF) [Key columns shown; note student info repeats per course]

enroll_id	stud_id	stud_name	course_code	course_title	sem_id	dept_id	dept_name	instr_name	grade	{prereqs}
E01	S01	Ali Hassan	CS101	Intro to CS	SEM1	D01	CS Dept	Dr. Noor	A	CS102:OOP
E02	S01	Ali Hassan	MATH 201	Calculus II	SEM1	D01	CS Dept	Prof. Zaid	B	MATH101:Calc I
E03	S02	Sara Khan	CS101	Intro to CS	SEM1	D01	CS Dept	Dr. Noor	A-	CS102:OOP
E04	S02	Sara Khan	CS201	OOP	SEM1	D01	CS Dept	Dr. Noor	NULL	CS101:Intro CS

Notice: Ali Hassan (S01) appears twice with the same dept_id/dept_name redundancy. {prereqs} is a repeating group violates 1NF.

2.2 First Normal Form (1NF)

Rule applied: Remove all repeating groups. Ensure every attribute is atomic (single-valued). Each row must be uniquely identifiable.

Action taken: Expand {prerequisites} into a separate flat row per prerequisite. The composite key for that table becomes (course_id, prereq_course_id).

Result — 1NF relations:

Relation	Attributes (PK underlined, FK in italics)
ENROLLMENT_REPORT_1NF	enrollment_id (PK), student_id, student_name, student_email, student_program, dept_id, dept_name, faculty_id, faculty_name, offering_id, course_code, course_title, credit_hours, course_description, semester_id, semester_name, reg_start, reg_end, room_id, building, room_number, capacity, instructor_id, instructor_name, instructor_email, instructor_type, day_of_week, start_time, end_time, grade, status
COURSE_PREREQ_1NF	course_code (PK), prereq_course_code (PK) ← composite key, course_title, prereq_title

Key observation: ENROLLMENT_REPORT_1NF has enrollment_id as PK but many non-key attributes depend only on student_id, course_code, semester_id, etc. — not on enrollment_id. This means partial dependencies exist → must move to 2NF.

ENROLLMENT_REPORT_1NF [Repeating group removed; all atomic]

enroll_id	stud_id	stud_name	course_code	course_title	sem_id	dept_id	dept_name	instr_name	grade	status
E01	S01	Ali Hassan	CS101	Intro to CS	SEM1	D01	CS Dept	Dr. Noor	A	Completed
E02	S01	Ali Hassan	MATH201	Calculus II	SEM1	D01	CS Dept	Prof. Zaid	B	Completed
E03	S02	Sara Khan	CS101	Intro to CS	SEM1	D01	CS Dept	Dr. Noor	A-	Completed
E04	S02	Sara Khan	CS201	OOP	SEM1	D01	CS Dept	Dr. Noor	NULL	Registered

Repeating {prereqs} group is eliminated. Still has redundancy (student/dept info repeating per row) — partial dependencies remain, addressed in 2NF.

COURSE_PREREQ_1NF [Separate relation for prerequisites]

course_code (PK)	prereq_course_code (PK)	course_title	prereq_title
MATH201	MATH101	Calculus II	Calculus I
CS201	CS101	OOP	Intro to CS
CS301	CS201	Data Structures	OOP

Each prerequisite is now its own atomic row with a composite PK (course_code, prereq_course_code).

2.3 Second Normal Form (2NF)

Rule applied: Every non-key attribute must be fully functionally dependent on the entire primary key. Remove all partial dependencies (attributes that depend on only part of a composite key, or on a non-PK functional determinant).

The PK of ENROLLMENT_REPORT_1NF is enrollment_id (single-attribute PK), but the UNF source contained functional dependencies that reveal hidden partial dependencies on sub-keys like student_id, course_code, semester_id. We decompose based on these functional dependencies:

Identified Functional Dependencies:

Determinant	Dependent Attributes
student_id	student_name, student_email, student_program, dept_id
dept_id	dept_name, faculty_id
faculty_id	faculty_name
course_code	course_title, credit_hours, course_description, dept_id
semester_id	semester_name, reg_start, reg_end
room_id	building, room_number, capacity
offering_id	course_code, semester_id, room_id
instructor_id	instructor_name, instructor_email, instructor_type
enrollment_id	student_id, offering_id, grade, status
offering_id + day_of_week + start_time	end_time (timetable)
offering_id + instructor_id	(TEACHES relationship — no extra attributes)

Resulting 2NF Relations:

Relation	Attributes
STUDENT	student_id (PK), student_name, student_email, student_program, dept_id (FK)
DEPARTMENT	dept_id (PK), dept_name, faculty_id (FK)
FACULTY_SCHOOL	faculty_id (PK), faculty_name
COURSE	course_code (PK), course_title, credit_hours, course_description, dept_id (FK)
SEMESTER	semester_id (PK), semester_name, reg_start, reg_end
ROOM	room_id (PK), building, room_number, capacity
COURSE_OFFERING	offering_id (PK), course_code (FK), semester_id (FK), room_id (FK)
INSTRUCTOR	instructor_id (PK), instructor_name, instructor_email, instructor_type, dept_id (FK)
TEACHES	offering_id (PK, FK), instructor_id (PK, FK) ← composite PK
TIMETABLE	timetable_id (PK), offering_id (FK), day_of_week, start_time, end_time
ENROLLMENT	enrollment_id (PK), student_id (FK), offering_id (FK), grade, status
COURSE_PREREQUISITE	course_code (PK, FK), prereq_course_code (PK, FK)

All relations are now in 2NF. Every non-key attribute is fully functionally dependent on its primary key. However, we must check for transitive dependencies → 3NF.

STUDENT

student_id (PK)	student_name	student_email	student_program	dept_id (FK)
S01	Ali Hassan	ali@uni.edu	BSCS	D01
S02	Sara Khan	sara@uni.edu	BSCS	D01
S03	Usman Ali	usman@uni.edu	BSIT	D02

COURSE

course_code (PK)	course_title	credit_hours	dept_id (FK)
CS101	Intro to CS	3	D01
CS201	OOP	3	D01
MATH201	Calculus II	3	D02

DEPARTMENT

dept_id (PK)	dept_name	faculty_id (FK)
D01	Computer Science	F01
D02	Mathematics	F01
D03	Physics	F02

SEMESTER

semester_id (PK)	semester_name	reg_start	reg_end
SEM1	Fall 2024	2024-08-01	2024-08-15
SEM2	Spring 2025	2025-01-05	2025-01-20

ENROLLMENT

enrollment_id (PK)	student_id (FK)	offering_id (FK)	grade	status
E01	S01	O01	A	Completed
E02	S01	O02	B	Completed
E03	S02	O01	A-	Completed
E04	S02	O03	NULL	Registered

Student info no longer repeats here — it lives in STUDENT table. Partial dependencies eliminated.

Each table above holds only attributes that depend on its own primary key. All partial dependencies from the 1NF flat table have been resolved.

2.4 Third Normal Form (3NF)

Rule applied: Remove all transitive dependencies a non-key attribute must not depend on another non-key attribute.

Checking each 2NF relation for transitive dependencies:

Relation	Transitive Dependency Found?	Action
STUDENT	dept_id → dept_name would be transitive if dept_name were here — it is NOT (dept_name is in DEPARTMENT). No violation.	None needed
DEPARTMENT	faculty_id → faculty_name would be transitive — but faculty_name is in FACULTY_SCHOOL, not here. No violation.	None needed
COURSE_OFFERING	No non-key depends on another non-key. offering_id → course_code, semester_id, room_id are all FKs.	None needed
INSTRUCTOR	instructor_type is atomic and depends directly on instructor_id. No transitive dependency.	None needed
TIMETABLE	start_time → end_time? No — end_time depends on the full record (offering + day + start), not on start_time alone. No violation.	None needed
ENROLLMENT	grade and status depend directly on enrollment_id (= student + offering context). No violation.	None needed
TEACHES	Composite PK, no non-key attributes. No violation.	None needed
COURSE_PREREQUISITE	Composite PK only. No non-key attributes. No violation.	None needed
ROOM	No transitive deps among building, room_number, capacity.	None needed
SEMESTER	reg_start, reg_end depend on semester_id. No transitive deps.	None needed

Conclusion: All 2NF relations are already in 3NF. No transitive dependencies were found because each decomposed relation carries only directly dependent attributes. The 2NF decomposition was thorough enough to eliminate transitive paths.

3NF Result — Same Schema as 2NF (No Transitive Dependencies Found)

All 2NF relations are already in 3NF. The sample data tables remain identical no further decomposition was required. Each non-key attribute depends only on the primary key, not on another non-key attribute.

DEPARTMENT (verified 3NF — no transitive dep.)

dept_id (PK)	dept_name	faculty_id (FK)
D01	Computer Science	F01
D02	Mathematics	F01

Example check: dept_name depends on dept_id (PK) directly. faculty_id is a FK, not carrying faculty_name here (that lives in FACULTY_SCHOOL). No transitive path exists.

2.5 Final Normalized Schema (3NF)

Below is the complete, final normalized relational schema with all primary keys, foreign keys, and constraints. (PK = Primary Key, FK = Foreign Key, NN = Not Null, UQ = Unique)

Table	Column	Type	Constraints
FACULTY_SCHOOL	faculty_id	INT	PK, NN
	name	VARCHAR(100)	NN
DEPARTMENT	dept_id	INT	PK, NN
	dept_name	VARCHAR(100)	NN
	faculty_id	INT	FK → FACULTY_SCHOOL, NN
	head_staff_id	INT	FK → STAFF, nullable
STAFF	staff_id	INT	PK, NN
	name	VARCHAR(100)	NN
	role	VARCHAR(50)	NN
	dept_id	INT	FK → DEPARTMENT,

Table	Column	Type	Constraints
			NN
INSTRUCTOR	instructor_id	INT	PK, NN
	name	VARCHAR(100)	NN
	email	VARCHAR(150)	UQ, NN
	instructor_type	ENUM('Full-time','Visiting')	NN
	dept_id	INT	FK → DEPARTMENT, nullable
STUDENT	student_id	INT	PK, NN
	name	VARCHAR(100)	NN
	email	VARCHAR(150)	UQ, NN
	program	VARCHAR(100)	NN
	dept_id	INT	FK → DEPARTMENT, NN
COURSE	course_code	VARCHAR(10)	PK, NN
	title	VARCHAR(150)	NN
	credit_hours	INT	NN, CHECK > 0
	description	TEXT	nullable
	dept_id	INT	FK → DEPARTMENT, NN
COURSE_PREREQUISITE	course_code	VARCHAR(10)	PK, FK → COURSE
	prereq_course_code	VARCHAR(10)	PK, FK → COURSE
SEMESTER	semester_id	INT	PK, NN
	name	VARCHAR(50)	NN
	reg_start	DATE	NN
	reg_end	DATE	NN

Table	Column	Type	Constraints
	grade_deadline	DATE	NN
ROOM	room_id	INT	PK, NN
	building	VARCHAR(100)	NN
	room_number	VARCHAR(20)	NN
	capacity	INT	NN, CHECK > 0
COURSE_OFFERING	offering_id	INT	PK, NN
	course_code	VARCHAR(10)	FK → COURSE, NN
	semester_id	INT	FK → SEMESTER, NN
	room_id	INT	FK → ROOM, NN
TEACHES	offering_id	INT	PK, FK → COURSE_OFFERING
	instructor_id	INT	PK, FK → INSTRUCTOR
TIMETABLE	timetable_id	INT	PK, NN
	offering_id	INT	FK → COURSE_OFFERING, NN
	day_of_week	ENUM('Mon','Tue','Wed','Thu','Fri','Sat')	NN
	start_time	TIME	NN
	end_time	TIME	NN
ENROLLMENT	enrollment_id	INT	PK, NN
	student_id	INT	FK → STUDENT, NN
	offering_id	INT	FK → COURSE_OFFERING, NN
	status	ENUM('Registered','Dr	NN

Table	Column	Type	Constraints
		opped','Completed')	
	grade	ENUM('A','B','C','D','F', 'I')	nullable

2.6 Normalization Summary

Normal Form	Requirement	Action Taken	Status
UNF	Flat, unstructured data	Identified all entities and repeating groups	Starting point
1NF	Atomic attributes, no repeating groups	Separated prerequisites into their own relation; ensured all columns are single-valued	Achieved
2NF	No partial dependencies on composite key	Decomposed large flat relation into 12 smaller relations, each grouped by its natural functional determinant	Achieved
3NF	No transitive dependencies	Verified all non-key attributes depend only on the PK no transitive paths found post-2NF decomposition	Achieved